Festival Optimiser

Build a console planner that recommends performances and compares two clash-free timetables.

Complete the prep HTML first. Allow 5–7 lessons of 90 minutes, or longer if needed. Finish all six stages; save a working version after each.

Stage 1 Store and display performances

Do this

1. Create five aligned 1D lists: names, genres, starts, ends and ratings. The same index must describe the same performance in every list.

2. Start with the six records below. Extend to at least 20 using festival_lists.txt or your own data.

3. Give each performance a stable ID, such as its original index + 1. Names can repeat.

4. Store times as whole minutes after midday. Every end must be later than its start. Display clock times: 90 means 13:30.

5. Display ID, name, genre, start, end and rating. Check equal list lengths.

ID	Name	Genre	Start	End	Rating
1	Neon Fox	indie	0	60	8
2	Orbit	rock	30	90	10
3	Pixel Hearts	pop	60	120	7
4	Static Bloom	indie	90	150	8
5	Echo Park	rock	120	180	6
6	Solar Tide	electronic	150	210	9

Check before moving on

- Orbit, ID 2: 12:30–13:30, rating 10.
- At least 20 records; duplicate names have different IDs.

Stage 2 Let the user change preferences

Do this

1. Build a repeating menu: display performances, change a rating, choose or clear a compulsory favourite, and exit.

2. Ask for an existing performance ID, then a whole-number rating from 0 to 10. Update only that performance.

3. Reject blank, text, decimal and out-of-range ratings. Explain the error and ask again. Reject unknown IDs.

4. Store one compulsory favourite by ID. It must appear in any generated timetable, even with a low rating. Allow replacement or clearing.

5. Write your own highest-rating search; do not use max(). Display the winning ID, name and rating.

6. State a tie rule, such as smaller original ID first. Use it consistently.

Check before moving on

- Starter data: highest-rated performance is Orbit, ID 2, rating 10.
- Set ID 3 to 9: only Pixel Hearts changes.
- Reject blank, seven, 7.5, −1 and 11; accept 0 and 10.
- Choosing and clearing a favourite both work.

Watch out A favourite identifies a performance, not just an artist name.

Stage 3 Sort and filter recommendations

Do this

1. Implement descending bubble sort or insertion sort yourself. Do not use sorted() or list.sort() for this assessed sort.

2. Keep records aligned and IDs unchanged. Move all fields together, or sort a separate list of original indices.

3. Show the top five performances, or all available if fewer than five. Apply your tie rule.

4. Add a genre filter. Ignore case and surrounding spaces; show up to five matches in rating order. Report no matches clearly.

5. Add both recommendation options to the menu.

Starter results

With smaller ID breaking ties: Orbit 10 → Solar Tide 9 → Neon Fox 8 → Static Bloom 8 → Pixel Hearts 7.

Check before moving on

- indie returns Neon Fox and Static Bloom; jazz returns no matches in the starter data.
- Changing a rating changes the order correctly.
- Test tied ratings and all ratings zero. Fields still match their original IDs.

Watch out Recommendations may clash. Stage 4 builds an attendable timetable.

Stage 4 Build a greedy timetable

Greedy chooses the highest remaining rating and keeps earlier choices.

Do this

1. Write and test a clash check. Sets meeting exactly at an endpoint do not clash.

2. Start with an empty selection. Add the compulsory favourite first, if chosen, and only once.

3. Consider remaining performances in descending rating order. Accept one only if it clashes with none already selected; otherwise skip it.

4. Display accepted performances in start-time order: ID, name, start, end and rating. Show the total rating.

5. Add a menu option. Use the current ratings and favourite each time.

Small test

A: 0–120, rating 10. B: 0–60, rating 6. C: 60–120, rating 6.

Check before moving on

- No favourite: greedy selects A, score 10.
- B compulsory: select B and C, score 12.
- Starter data, no favourite: Orbit, Static Bloom, Solar Tide; score 27.
- Test overlap, containment, identical times and back-to-back sets.

Watch out Check against every selected performance, not just the last.

Optional hint: clash when startA < endB AND startB < endA.

Stage 5 Find the optimum by exhaustive search

A subset is any selection, including none. Exhaustive search checks every subset and keeps the best valid timetable.

Do this

1. Start with three performances. Generate all eight include/exclude combinations, including the empty subset. Use recursion or bit masks; do not hand-write the combinations.

2. Reject a subset if any selected pair clashes or the compulsory favourite is missing.

3. For valid subsets, total the ratings and remember the best. State how equal scores are resolved.

4. Display the best selection in time order with its score. Count every candidate subset considered.

5. Add a menu option using 10–15 performances. If the favourite is outside this reduced dataset, report the problem. Keep exhaustive search off the full 20-record dataset.

Check before moving on

- Stage 4's A/B/C test: optimum B+C, score 12. With A compulsory: A only, score 10.
- Candidate counts: 3 records → 8; 5 → 32; 10 → 1,024; 15 → 32,768.
- Test all-zero ratings and a zero-rated compulsory favourite.

Watch out There are 2^n subsets. Python's enumerate() does not generate them.

Stage 6 Compare and submit evidence

Do this

1. Compare both schedulers on the same 5, 10 and 15 records, ratings and favourite at each size.

2. Record: dataset size, favourite ID or none, selected IDs and score for each method, greedy overlap checks and exhaustive subsets checked. Define each counter.

3. Include a case where greedy loses and one where it matches the optimum. Explain the results and why exhaustive search grows quickly.

4. Test the complete menu. Record expected and actual results for the checks below.

Required tests

Invalid ratings and IDs; ties; all-zero ratings; duplicate names; overlapping, identical and back-to-back intervals; favourite chosen, cleared or outside reduced input; no matching genre; changed ratings.

Submit

- Python code and README with run instructions and tie rules.
- Expected/actual tests, comparison table and explanation.
- 200–300 word reflection: actual bug, evidence, fix, two test results and one limitation. Record help or hints used.

Check before moving on

- All six stages work. Be ready to trace a fresh example and explain a small code change.

Watch out Overlap checks and subsets are different units. Do not treat their counts as identical work.

Optional stretch: add travel time, or investigate weighted interval scheduling with dynamic programming.